6️⃣

# Phase 6 - Audit Evidence Pack Assembly (Sanitized)

This phase completes **Lab 3B** by assembling the audit evidence pack required for APPI compliance submission.

> 🎯 **Objective:** Create the `audit-pack/` folder with all required proof files + auditor narrative to complete Lab 3 and finish Armageddon.

---

## What You Need to Deliver

### Deliverable A — Audit Evidence Pack Folder

```
audit-pack/
├── 00_architecture-summary.md
├── 01_data-residency-proof.txt
├── 02_edge-proof-cloudfront.txt
├── 03_waf-proof.txt
├── 04_cloudtrail-change-proof.txt
├── 05_network-corridor-proof.txt
└── evidence.json   (Malgus scripts output)
```

### Deliverable B — Auditor Narrative

An **8–12 line paragraph** explaining:

- Why this design is APPI-safe
- Why the DB cannot be placed overseas

---

# Step-by-Step: Creating Each Proof File

## File 1: `00_ architecture-summary.md`

**Purpose:** High-level overview of your multi-region architecture

**Content to include:**

- Tokyo (ap-northeast-1) = Data Authority (RDS lives here)
- São Paulo (sa-east-1) = Compute Spoke (NO RDS)
- Transit Gateway peering connects both regions
- PHI never leaves Tokyo

---

## File 2: `01_data-residency-proof.txt`

**Purpose:** Prove RDS exists ONLY in Tokyo

**CLI Commands:**

```
# Run this and save output to 01_data-residency-proof.txt

echo "=== DATA RESIDENCY PROOF ==="
echo "Generated: $(date)"
echo ""

echo "=== Tokyo RDS (should have helga-rdslab3) ==="
aws rds describe-db-instances --region ap-northeast-1 \
  --query "DBInstances[].{DB:DBInstanceIdentifier,AZ:Availabi
lityZone,Region:'ap-northeast-1',Endpoint:Endpoint.Address}"
\
  --output table

echo ""
echo "=== São Paulo RDS (should be EMPTY - APPI compliance) =
=="
aws rds describe-db-instances --region sa-east-1 \
```

```
  --query "DBInstances[].DBInstanceIdentifier" \
  --output table
```

**Expected Result:** Tokyo shows `helga-rdslab3`, São Paulo shows empty.

---

## File 3: `02_edge-proof-cloudfront.txt`

**Purpose:** Prove CloudFront is the edge layer (Hit/Miss/RefreshHit evidence)

**CLI Commands:**

```
# Option A: Check CloudFront headers
curl -I https://example.com/api/public-feed

# Option B: List CloudFront logs from S3
aws s3 ls s3://your-log-bucket/cloudfront-logs/ --recursive |
tail -n 20

# Option C: Download and inspect a log file
aws s3 cp s3://your-log-bucket/cloudfront-logs/<somefile>.gz
.
zcat <somefile>.gz | head -50
```

**What to capture:** Evidence of `x-edge-result-type` showing Hit/Miss/RefreshHit

---

## File 4: `03_waf-proof.txt`

**Purpose:** Prove WAF is filtering traffic (Allow vs Block)

**Options:**

1. WAF log snippet from CloudWatch Logs
2. WAF Insights summary
3. Output from `malgus_waf_summary.py` script

**CLI to check WAF exists:**

```
aws wafv2 list-web-acls --scope CLOUDFRONT --region us-east-1
\
  --query "WebACLs[].{Name:Name,Id:Id}"
```

---

## File 5: `04_cloudtrail-change-proof.txt`

**Purpose:** Prove who changed SG/TGW/WAF/CloudFront config

**CLI Commands:**

```
# Check recent CloudTrail events (90-day history available by
default)
aws cloudtrail lookup-events --region ap-northeast-1 \
  --lookup-attributes AttributeKey=EventName,AttributeValue=A
uthorizeSecurityGroupIngress \
  --max-results 10

aws cloudtrail lookup-events --region ap-northeast-1 \
  --lookup-attributes AttributeKey=EventName,AttributeValue=C
reateTransitGateway \
  --max-results 10
```

**What to capture:** Who made changes, when, and what was changed

---

## File 6: `05_network-corridor-proof.txt`

**Purpose:** Prove TGW attachments + routes form the legal corridor

**CLI Commands:**

```
echo "=== NETWORK CORRIDOR PROOF ==="
echo "Generated: $(date)"
echo ""

# TGW Peering Status
echo "=== TGW Peering (Tokyo view) ==="
```

```
aws ec2 describe-transit-gateway-peering-attachments --region
ap-northeast-1 \
  --query "TransitGatewayPeeringAttachments[*].{State:State,R
equesterTGW:RequesterTgwInfo.TransitGatewayId,AccepterTGW:Acc
epterTgwInfo.TransitGatewayId}"

echo ""
echo "=== TGW Peering (São Paulo view) ==="
aws ec2 describe-transit-gateway-peering-attachments --region
sa-east-1 \
  --query "TransitGatewayPeeringAttachments[*].{State:State}"

echo ""
echo "=== Tokyo Routes to São Paulo ==="
aws ec2 describe-route-tables --region ap-northeast-1 \
  --filters "Name=tag:Name,Values=*helga*private*" \
  --query "RouteTables[*].Routes[?DestinationCidrBlock=='10.
1.0.0/16']"

echo ""
echo "=== São Paulo Routes to Tokyo ==="
aws ec2 describe-route-tables --region sa-east-1 \
  --filters "Name=tag:Name,Values=*liberdade*private*" \
  --query "RouteTables[*].Routes[?DestinationCidrBlock=='10.
0.0.0/16']"

echo ""
echo "=== RDS SG Rules (should include 10.1.0.0/16) ==="
aws ec2 describe-security-groups --region ap-northeast-1 \
  --filters "Name=group-name,Values=*rds*" \
  --query "SecurityGroups[*].IpPermissions[?FromPort==\`3306
\`].IpRanges"
```

**Expected:** Peering = `available`, routes = `active`, SG includes São Paulo CIDR

### File 7: `evidence.json`

**Purpose:** Consolidated machine-readable output from Malgus scripts

**Scripts to run:**

```
# Script 1 - DB residency proof
python3 malgus_residency_proof.py

# Script 2 - TGW corridor proof
python3 malgus_tgw_corridor_proof.py

# Script 3 - CloudTrail changes
python3 malgus_cloudtrail_last_changes.py

# Script 4 - WAF summary
python3 malgus_waf_summary.py

# Script 5 (optional) - CloudFront log analysis
python3 malgus_cloudfront_log_explainer.py --latest 5
```

Combine outputs into `evidence.json`.

---

## Deliverable B: Auditor Narrative Template

Write 8–12 lines covering:

```
## Auditor Narrative

This architecture ensures APPI compliance by enforcing strict
data residency:

1. **Data Residency:** The RDS database (helga-rdslab3) exist
s exclusively in
   Tokyo (ap-northeast-1). No database instances exist in São
Paulo or any
   other region.
```

```
2. **Compute Separation:** São Paulo (sa-east-1) provides com
pute capacity
   only. It has no persistent storage for PHI.

3. **Controlled Corridor:** Transit Gateway peering creates a
verified network
   path between regions. All cross-region traffic traverses t
his controlled
   corridor.

4. **Why DB Cannot Be Overseas:** Under APPI, personal inform
ation of Japanese
   residents must remain within Japan unless explicit consent
is obtained for
   overseas transfer. By keeping the RDS in Tokyo, we ensure
PHI never leaves
   Japanese jurisdiction.

5. **Audit Trail:** CloudTrail provides immutable 90-day even
t history. All
   infrastructure changes are logged and attributable.
```

## Completion Checklist

- [ ] Created `audit-pack/` folder
- [ ] `00_architecture-summary.md` — Architecture overview
- [ ] `01_data-residency-proof.txt` — RDS only in Tokyo
- [ ] `02_edge-proof-cloudfront.txt` — CloudFront cache evidence
- [ ] `03_waf-proof.txt` — WAF Allow/Block summary
- [ ] `04_cloudtrail-change-proof.txt` — Change audit trail
- [ ] `05_network-corridor-proof.txt` — TGW peering + routes

- [ ] `evidence.json` — Malgus scripts output
- [ ] Auditor Narrative (8–12 lines)

> 🏁 **When all boxes are checked, Lab 3B is COMPLETE and Armageddon is FINISHED.**

---

# Running the Malgus Scripts

## Step 1: Create a folder for the scripts

```
mkdir -p ~/lab3/audit-pack/scripts
cd ~/lab3/audit-pack/scripts
```

## Step 2: Make sure Python and boto3 are installed

```
# Check Python 3
python3 --version

# Install boto3 if you don't have it
pip3 install boto3
```

## Step 3: Make sure AWS CLI is configured

```
# Verify you're authenticated
aws sts get-caller-identity
```

If this returns your account info, you're good. If not, run `aws configure`.

## Step 4: Run each script

```
cd ~/audit-pack/scripts
```

```
python malgus_residency_proof.py
python malgus_tgw_corridor_proof.py
python malgus_cloudtrail_last_changes.py
python malgus_waf_summary.py
python malgus_cloudfront_log_explainer.py --latest 5
```

## Step 5: Save outputs to evidence.json

```
python malgus_residency_proof.py > residency.json
python malgus_tgw_corridor_proof.py > tgw.json
python malgus_cloudtrail_last_changes.py > cloudtrail.json
python malgus_waf_summary.py > waf.json
```